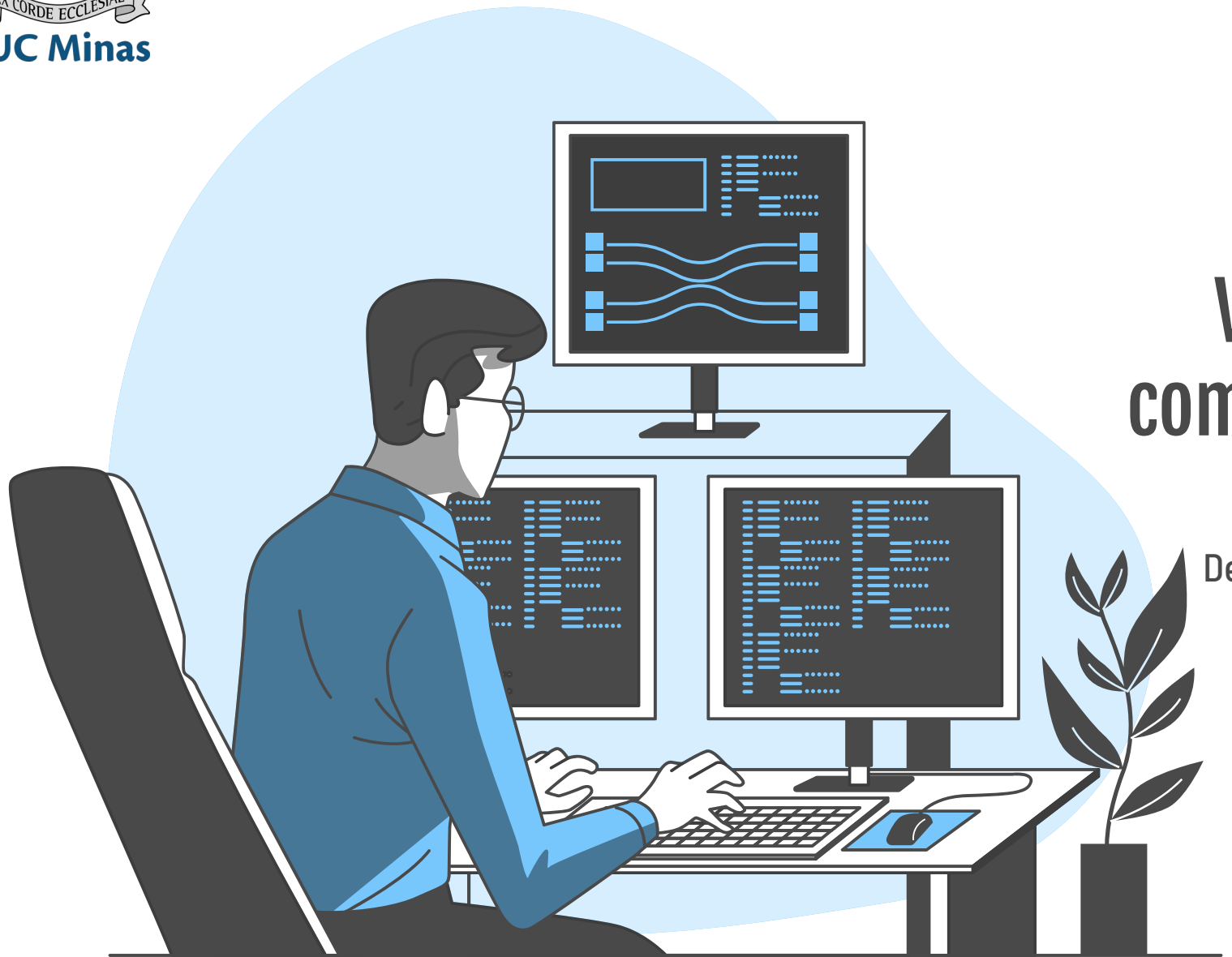
PUC Minas



Aula 01 · Deploy

Visão geral do deploy: do computador para a internet

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1. Entender **build**, **deploy** e **ambiente**
2. Saber por que o front vai para o **Vercel** e o back para o **Render**
3. Mapear as **variáveis de ambiente** de cada serviço
4. Conhecer os limites dos **planos gratuitos**
5. Seguir a **ordem recomendada** de publicação

Antes de começar

- Projeto full stack funcionando localmente (pasta FullStack)
- Repositório no GitHub
- Contas no Vercel e no Render (login com GitHub)

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 O que é deploy

02 Quem hospeda o quê

03 Ambientes e variáveis

04 Planos gratuitos: o que esperar

05 Ordem recomendada

06 Antes de começar: checklist do repositório

07 Aprofundando: conceitos de hospedagem

01

O que é deploy

O que é deploy

Deploy é colocar a aplicação para rodar em um servidor acessível pela internet. Envolve duas etapas: o **build** (transformar o código em algo executável: o `dist/` do Vite, o `.jar` do Spring) e a **execução** em um servidor, com as **variáveis de ambiente** de produção.

Os serviços que usamos fazem isso automaticamente a partir do **GitHub**: a cada `git push`, eles baixam o código, fazem o build e publicam a nova versão. Isso se chama **deploy contínuo**.

01

O que é deploy

Aula 01 · Deploy

O que é deploy



Um push, dois deploys: cada serviço publica a sua pasta do mesmo repositório.

02

Quem hospeda o quê

Quem hospeda o quê

Parte	Serviço	Por quê	Resultado
Front (React + Vite)	**Vercel**	O build é só HTML/CSS/JS estático; o Vercel serve por uma CDN global, rápido e sempre ligado	`https://meu-app.vercel.app`
Back (Spring Boot)	**Render**	Precisa de um processo Java rodando o tempo todo; o Render roda containers **Docker**	`https://meu-app-api.onrender.com`
Banco relacional	**Supabase**	PostgreSQL gerenciado	*connection string* JDBC
Banco não relacional	**MongoDB Atlas**	MongoDB gerenciado	`mongodb+srv://...`

Quem hospeda o quê

O Vercel não roda Spring Boot (ele executa funções *serverless* em Node, Python, Go, não um servidor Java contínuo). Por isso o back vai para o Render, que aceita qualquer coisa empacotada em um **Dockerfile** (aulas 02 a 04).

02

Quem hospeda o quê

Aula 01 · Deploy

03

Ambientes e variáveis

Ambientes e variáveis

O mesmo código roda em lugares diferentes. O que muda entre eles fica em **variáveis de ambiente**, nunca no código:

Variável	Onde cadastrar	Valor local	Valor em produção
`VITE_API_URL`	**Vercel**	*(vazio → `localhost:8080`)*	`https://meu-app-api.onrender.com`
`VITE_SUPABASE_URL` / `..._PUBLISHABLE_KEY`	**Vercel**	`.env.local`	mesmos valores
`MONGODB_URI`	**Render**	`mongodb://localhost:27017` ou Atlas	`mongodb+srv://usuario:senha@cluster0...`
`DB_URL`, `DB_USER`, `DB_PASSWORD`	**Render**	Supabase / Postgres local	Supabase (*Session pooler*)
`CORS_ORIGINS`	**Render**	`http://localhost:5173`	`https://meu-app.vercel.app`
`JWT_SECRET`	**Render**	valor de desenvolvimento	segredo gerado com `openssl rand -base64 64`
`PORT`	*(o Render define sozinho)*	`8080`	`10000` (padrão do Render)

Ambientes e variáveis

Repare no cruzamento: o **Vercel** precisa saber a URL do **Render** (`VITE_API_URL`) e o **Render** precisa saber a URL do **Vercel** (`CORS_ORIGINS`). É esse “apontar um para o outro” que a aula 07 detalha.

03

Ambientes e variáveis

Aula 01 · Deploy

04

Planos gratuitos: o que esperar

Planos gratuitos: o que esperar

Serviço	Limite que mais afeta você
Render (Free)	O serviço **dorme após 15 minutos sem acesso**; a primeira requisição depois disso leva cerca de 50 segundos. 512 MB de RAM. 750 horas/mês.
Vercel (Hobby)	Uso pessoal e não comercial; limites generosos de banda para projetos de aula.
Supabase (Free)	Projeto pausado após **7 dias** sem uso; 2 projetos ativos; 500 MB de banco.
MongoDB Atlas (Free)	512 MB; cluster compartilhado; pode ser pausado após longo período sem uso.

Planos gratuitos: o que esperar

Dica

Vai apresentar o trabalho? **Acorde o back 2 minutos antes** abrindo `https://seu-back.onrender.com/api/status` no navegador e entre no painel do Supabase para garantir que o projeto não está pausado.

05

Ordem recomendada

Ordem recomendada

Siga sempre esta ordem. Cada passo produz uma informação que o próximo precisa:

1. **Banco** (Atlas ou Supabase): crie o cluster/projeto, o usuário e libere o acesso de rede. Resultado: a *connection string*.
2. **Docker local**: confira que o `Dockerfile` do back faz o build e sobe (aula 03). Erros aqui aparecerão no Render também.
3. **Back no Render** com a *connection string*. Resultado: a URL `https://...onrender.com`. Teste `/api/status`.
4. **Front no Vercel** com `VITE_API_URL` = URL do Render. Resultado: a URL `https://...vercel.app`.
5. **Volte ao Render** e coloque a URL do Vercel em `CORS_ORIGINS`. Salve (o Render faz novo deploy).

Ordem recomendada (cont.)

6

Teste o fluxo completo pelo site do Vercel, com o DevTools aberto.

06

Antes de começar: checklist do repositório

Antes de começar: checklist do repositório

Verificações antes do primeiro deploy

```
git status                              # nada pendente
git ls-files | grep -E "node_modules|target/|\.env\.local"   # deve sair VAZIO
grep -rn "mongodb+srv://" Back/src  # NÃO pode ter senha real no código
cd Front && npm run build           # o build do front passa localmente?
cd ../Back && ./mvnw clean package  # o build do back passa localmente?
```

Antes de começar: checklist do repositório

Atenção

O Render e o Vercel fazem o build em uma máquina Linux **limpa**. Diferenças comuns em relação ao seu computador: **maiúsculas e minúsculas importam** (`import Header from "./header"` funciona no Windows e quebra no Linux se o arquivo for `Header.jsx`), o `mvnw` precisa de permissão de execução e só existe o que está **no GitHub**.

07

Aprofundando: conceitos de hospedagem

Modelos de hospedagem

IaaS

Você recebe uma máquina virtual e instala tudo (AWS EC2, Azure VM).

PaaS

Você entrega o código ou o Dockerfile e o serviço roda (**Render**, Railway, Fly.io).

Hospedagem estática / CDN

Você entrega HTML, CSS e JS prontos (**Vercel**, Netlify, Cloudflare Pages).

DBaaS

Banco gerenciado: backup, atualização e acesso pela internet (**Atlas**, **Supabase**).

Site estático x servidor de aplicação

Estático (front no Vercel)

- Arquivos copiados para servidores no mundo todo (CDN)
- Sempre ligado, sem cold start
- Não guarda segredos nem acessa banco com senha

vs

Servidor (back no Render)

- Um processo Java rodando o tempo todo
- Executa regras, guarda segredos, fala com o banco
- No plano gratuito dorme e acorda

Aprofundando: conceitos de hospedagem

Ambiente	Quem usa	Banco	URL típica
Desenvolvimento	Você, no seu computador	Local (Docker) ou Atlas/Supabase de teste	`localhost:5173` / `localhost:8080`
Preview / homologação	Equipe e professor, para revisar	Banco de teste	`app-git-feature.vercel.app`
Produção	Usuários reais	Banco oficial	`app.vercel.app` / domínio próprio

DNS e HTTPS em uma página

O **DNS** traduz um nome (`app.vercel.app`) para o endereço IP do servidor. O **HTTPS** criptografa a conversa entre o navegador e o servidor com um certificado digital. Vercel e Render emitem certificados gratuitos automaticamente, inclusive para domínios próprios, então você nunca precisa configurar HTTPS à mão.

07

Aprofundando: conceitos de hospedagem

Aula 01 · Deploy

DNS e HTTPS em uma página

Termo	Significado
Build	Transformar código-fonte em artefato executável (`dist/`, `.jar`, imagem Docker)
Artefato	O resultado do build, que vai para o servidor
Deploy	Colocar o artefato para rodar
Rollback	Voltar para a versão anterior quando a nova falha
Cold start	Tempo para um serviço adormecido voltar a responder
Health check	Endpoint que o serviço consulta para saber se a app está saudável

Aprofundando: conceitos de hospedagem

Dica

Escreva no README do projeto uma tabela com as **URLs de produção** (front, back, `/api/status`) e as **variáveis de ambiente** de cada serviço. Quem for avaliar ou continuar o projeto agradece, e você também, daqui a seis meses.

Aprofundando: conceitos de hospedagem

1. Confirme que `npm run build` e `./mvnw clean package` passam localmente.
2. Confirme que o repositório no GitHub não tem segredos nem pastas geradas.
3. Prepare o banco na nuvem e anote a connection string em lugar seguro.
4. Siga a ordem: Render → Vercel → CORS no Render.
5. Teste pelo site publicado com o DevTools aberto.

08

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

Por que o Spring Boot não vai para o Vercel?

A. O Vercel não aceita Java como processo contínuo; ele serve sites estáticos e funções serverless

B. O Vercel é pago

C. O Spring não tem build

D. Por causa do CORS

Resposta

Por que o Spring Boot não vai para o Vercel?

✓ O Vercel não aceita Java como processo contínuo; ele serve sites estáticos e funções serverless

B O Vercel é pago

C O Spring não tem build

D Por causa do CORS

Por quê: O back roda em container Docker no Render.

Pergunta

Qual variável o Vercel precisa para achar o back?

A CORS_ORIGINS

B VITE_API_URL

C MONGODB_URI

D PORT

Resposta

Qual variável o Vercel precisa para achar o back?

A CORS_ORIGINS

✓ VITE_API_URL

C MONGODB_URI

D PORT

Por quê: Ela recebe a URL do Render.

Pergunta

Qual a ordem recomendada de publicação?

A Vercel → banco → Render

B Banco → Render → Vercel → CORS no Render

C Render → banco → Vercel

D Qualquer ordem

Resposta

Qual a ordem recomendada de publicação?

A. Vercel → banco → Render

✓ Banco → Render → Vercel → CORS no Render

C. Render → banco → Vercel

D. Qualquer ordem

Por quê: Cada passo gera o dado que o próximo precisa.

Pergunta

O que acontece com o Render Free sem acessos por 15 minutos?

A. É apagado

B. Dorme e leva ~50 s para acordar

C. Perde as variáveis

D. Muda de URL

Resposta

O que acontece com o Render Free sem acessos por 15 minutos?

A. É apagado

✓ Dorme e leva ~50 s para acordar

C. Perde as variáveis

D. Muda de URL

Por quê: Acorde antes de apresentar.

Pergunta

Por que um import com maiúscula errada quebra só no deploy?

A Bug do Vercel

B O build roda em Linux, que diferencia maiúsculas de minúsculas

C Por causa do CORS

D Falta o Node

Resposta

Por que um import com maiúscula errada quebra só no deploy?

A Bug do Vercel

✓ O build roda em Linux, que diferencia maiúsculas de minúsculas

C Por causa do CORS

D Falta o Node

Por quê: No Windows o nome errado funciona.

Fechamento

Resumo da aula

- ✓ Deploy = build + execução com variáveis de produção; Vercel e Render fazem isso a cada `git push`.
- ✓ Front estático no **Vercel**; back Java em container **Docker** no **Render**; bancos no Atlas/Supabase.
- ✓ Variáveis: `VITE_API_URL` no Vercel; `MONGODB_URI`/`DB_*`, `CORS_ORIGINS`, `JWT_SECRET` no Render.
- ✓ Render Free dorme após 15 min; Supabase Free pausa após 7 dias.
- ✓ Ordem: banco → Docker local → Render → Vercel → CORS no Render → teste.

Exercícios

1. Monte a tabela de variáveis do seu projeto com valores locais e de produção (sem senhas reais).

2. Rode o checklist do repositório e corrija o que falhar.

3. Explique com suas palavras por que o Spring Boot não vai para o Vercel.

4. Desenhe a arquitetura de produção do seu projeto com as URLs finais.

Para ir além

- render.com/docs/free
- vercel.com/docs/plans/hobby
- Seção “Deploy” do README de `FullStack/PalpiteDTI`

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 01?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web